Lista de exercícios – Listas

Orientações para resolução da lista

- Esta lista trabalha exclusivamente o conteúdo de **listas**: criação, acesso por índice, inclusão de elementos com `append`, percurso com `for` (por elemento e por índice), percurso com `while` e as funções `len`, `min`, `max` e `sum`.
- Todas as questões são de **construção de código**. Antes de implementar qualquer solução, procure identificar claramente as entradas, o processamento necessário e as saídas esperadas.
- Lembre-se de que os índices começam em `0` e de que o último índice válido é `len(lista) - 1`. Acessar uma posição inexistente provoca erro.
- Use o percurso por elemento quando o foco estiver no valor e o percurso por índice quando a posição também for importante.
- Tente implementar a sua própria lógica antes de recorrer a funções prontas. Quando o enunciado proibir uma função, respeite a restrição.
- Teste cada solução com os exemplos do enunciado e elabore também os seus próprios casos de teste, incluindo listas vazias ou com um único elemento quando fizer sentido.
- Desenvolva as soluções individualmente. O objetivo da lista é consolidar seu aprendizado e desenvolver autonomia na construção de programas.

Exercícios

1. Faça um programa que leia um número inteiro N e, em seguida, N números inteiros, armazenando-os em uma lista. Depois de armazenar todos os valores, o programa deve exibir os números na ordem inversa da leitura, um por linha, e informar separadamente a soma dos valores que ocupam posições pares e a soma dos valores que ocupam posições ímpares da lista.

Por exemplo, para a lista:

```
[7, 3, 9, 1, 8]
```

os valores devem ser exibidos na ordem 8, 1, 9, 3, 7 e a saída deve indicar ainda:

```
Soma das posicoes pares: 24
Soma das posicoes impares: 4
```

Observe que a paridade considerada é a do índice, e não a do valor armazenado.

2. Crie uma função chamada `filtrar_palavras(palavras, tamanho)` que receba uma lista de palavras e um número inteiro e retorne uma nova lista contendo apenas as palavras que possuem mais caracteres do que o tamanho informado, preservando a ordem original.

Por exemplo:

```
filtrar_palavras(["sol", "computador", "lista", "python"], 5)
-> ["computador", "python"]

filtrar_palavras(["a", "de", "para"], 4)
-> []
```

No programa principal, leia palavras digitadas pelo usuário até que ele digite `fim`, armazene-as em uma lista, leia o tamanho desejado e exiba a lista retornada pela função e a quantidade de palavras que ela contém.

3. Crie uma função chamada `alterna_paridade(valores)` que receba uma lista de números inteiros e retorne `True` se os valores alternam entre par e ímpar ao longo de toda a lista, ou `False` caso contrário.

A lista pode começar tanto por um valor par quanto por um valor ímpar. O que importa é que dois vizinhos nunca tenham a mesma paridade.

Por exemplo:

```
[2, 7, 4, 9, 6] -> True
[5, 8, 3, 10]   -> True
[2, 7, 9, 4]    -> False
[6]             -> True
```

4. Crie uma função chamada `sem_repeticao(valores)` que receba uma lista de números inteiros e retorne uma nova lista na qual cada valor aparece uma única vez. Deve ser mantida apenas a **primeira** ocorrência de cada valor, na ordem em que aparece na lista original.

Por exemplo:

```
[4, 7, 4, 2, 7, 7, 9, 2]
```

deve produzir:

```
[4, 7, 2, 9]
```

Para decidir se um valor já foi incluído no resultado, percorra a lista que está sendo construída. A lista recebida como parâmetro não deve ser modificada.

5. Crie uma função chamada `segundo_maior(valores)` que receba uma lista de números inteiros com pelo menos dois valores distintos e retorne o segundo maior valor da lista. Valores iguais ao maior não devem ser considerados como segundo maior.

Por exemplo:

```
[12, 7, 25, 3, 18] -> 18
[9, 9, 4, 9]       -> 4
[1, 2]             -> 1
```

A função deve percorrer a lista sem modificá-la. Não utilize `max()`, `min()`, `sorted()` nem o método `sort()`.

6. Escreva uma função chamada `maior_diferenca(valores)` que receba uma lista de números inteiros com pelo menos dois elementos e retorne a maior diferença encontrada entre dois valores **consecutivos** da lista.

A diferença deve ser sempre considerada como um valor positivo, independentemente de a sequência estar subindo ou descendo.

Por exemplo:

```
[10, 4, 7, 15, 13] -> 8
[5, 5, 5]          -> 0
[20, 3]            -> 17
```

No primeiro caso, as diferenças entre vizinhos são 6, 3, 8 e 2. Não utilize as funções `max()` e `abs()`.

7. Faça um programa que leia um número inteiro N e, em seguida, N notas, armazenando-as em uma lista. Depois de armazenar todos os valores, o programa deve calcular a média das notas e construir duas novas listas: uma com as notas acima da média e outra com as notas abaixo da média. Notas iguais à média não pertencem a nenhuma das duas listas.

Ao final, exiba a média, as duas listas e a posição (índice) da maior nota informada. Caso a maior nota apareça mais de uma vez, considere a primeira ocorrência.

Por exemplo, para as notas:

```
[6.0, 9.0, 4.0, 9.0, 7.0]
```

a saída deve indicar:

```
Media: 7.0
Acima: [9.0, 9.0]
Abaixo: [6.0, 4.0]
Posicao da maior: 1
```

Não utilize as funções `sum()` e `max()`.

8. Uma lista é considerada **espelhada** quando seus elementos são os mesmos se lidos do início para o final ou do final para o início.

Crie uma função chamada `eh_espelhada(valores)` que receba uma lista e retorne `True` se ela for espelhada ou `False` caso contrário.

Por exemplo:

```
[1, 4, 9, 4, 1] -> True
[7, 2, 2, 7]    -> True
[3, 5, 8, 3]    -> False
[6]             -> True
[]              -> True
```

A função deve comparar os elementos utilizando índices, sem construir uma segunda lista e interrompendo a verificação assim que uma diferença for encontrada.

9. Crie uma função chamada `mesclar(lista1, lista2)` que receba duas listas de números inteiros, cada uma delas **já organizada em ordem crescente**, e retorne uma nova lista contendo todos os elementos das duas, também em ordem crescente.

Por exemplo:

```
lista1 = [2, 5, 9, 14]
lista2 = [1, 5, 6]
```

deve produzir:

```
[1, 2, 5, 5, 6, 9, 14]
```

A cada passo, compare o próximo elemento ainda não utilizado de cada lista e acrescente o menor deles ao resultado. Quando uma das listas terminar, os elementos restantes da outra devem ser acrescentados ao final.

As listas recebidas não devem ser modificadas. Não utilize `sorted()` nem o método `sort()`.

10. Um sistema de análise de desempenho acompanha uma prova de corrida. Os dados dos participantes são armazenados em duas listas de mesmo tamanho: a primeira contém os nomes dos atletas e a segunda contém os tempos, em segundos, obtidos por cada um deles. O atleta da posição `i` da primeira lista é o dono do tempo da posição `i` da segunda.

Por exemplo:

```
nomes  = ["Ana", "Bruno", "Carla", "Davi", "Eva"]
tempos = [12.5, 11.8, 13.2, 10.9, 12.0]
```

Faça um programa que leia a quantidade de atletas e, para cada um, seu nome e seu tempo. Em seguida, o programa deve:

a) calcular e exibir o tempo médio da prova;

b) exibir os nomes dos atletas cujo tempo ficou abaixo da média;

c) exibir o pódio da prova, isto é, o nome e o tempo dos três atletas mais rápidos, do primeiro ao terceiro colocado.

Para o exemplo acima, a média é 12.08, os atletas abaixo da média são Bruno, Davi e Eva, e o pódio é formado por Davi, Bruno e Eva, nessa ordem.

Para montar o pódio, percorra as listas localizando o menor tempo ainda não escolhido. Uma lista auxiliar com as posições já utilizadas pode ajudar. Considere que existem pelo menos três atletas e que não há tempos repetidos. Não utilize `sorted()`, `sort()` nem `min()`.